

Tema 1 – Liste înlanțuite, stive, cozi

Răzvan Șerb, Alexandru Tudor, Andreea Duțulescu, Cătălin Rîpanu,
Ștefan Turcu, Andrei Voicu, Octavian Palaga

Data postării: 20.03.2026

Deadline: 15.04.2026 ora 23:59

Sistemele moderne de gestionare a apelurilor de urgență reprezintă o componentă esențială a infrastructurii de siguranță publică. În România, sistemul unic pentru apeluri de urgență 112 este sub administrația Serviciului de Telecomunicații Speciale (STS), permițându-le cetățenilor să raporteze situații precum accidente, incendii sau urgențe medicale.

Odată cu extinderea și dezvoltarea populației, complexitatea și volumul situațiilor de urgență au crescut semnificativ. Într-un sistem simplist, apelurile ar putea fi preluate și apoi soluționate strict în ordinea preluării (primul-venit, primul-servit). Dar considerând circumstanțe realiste, dispeceratele se confruntă cu zeci de apeluri simultane prin intermediul cărora sunt raportate incidente cu diferite grade de severitate, așadar o abordare liniară s-ar dovedi insuficientă.

1 Descriere problemă

În această temă, vom explora o arhitectură pentru gestionarea apelurilor de urgență 112 inspirată din sistemele de dispecerizare reale. În continuare va fi descrisă o serie de elemente care vor ajuta la crearea unei simulări în care incidentele raportate prin apeluri de urgență 112 sunt organizate pe niveluri de prioritate și alocarea numărului limitat de echipaje de intervenție este realizată în mod automat.

Pentru a putea remedia incidentele raportate, sistemul dispune de echipaje de intervenție (spre exemplu: echipaje de poliție, mașini de pompieri sau ambulante). În cadrul acestei teme, se va defini un echipaj de intervenție printr-o **structură** ce conține: un ID unic (nr. întreg), tipul (caracter: 'A'/'B'/'C') și disponibilitatea (nr. întreg: 0=unavailable / 1=available).

```
struct unit {  
    ID  
    tipul echipajului  
    disponibilitate  
    int id;  
    char type;  
    int availability;  
};
```

int id;
char type;
int availability;

Important

Numărul echipajelor de intervenție și datele acestora vor fi citite din fișierul de intrare. **Vor exista maximum 50 de echipaje de intervenție și le puteți salva cum doriți (spre exemplu vector de structuri sau listă).**

Un incident este o situație de urgență care necesită efectuarea unei acțiuni de remediere. În această temă, se va defini un incident printr-o **structură** ce include: un ID unic (nr. întreg), gradul de prioritzare - severitatea incidentului (șir de caractere: "low"/"medium"/"high"), descrierea (șir de caractere) și status-ul (șir de caractere: "queued"/"intervened"/"solved").

```
struct incident {  
    ID          int id;  
    grad de prioritzare char priority[7];  
    descriere    char *description;  
    status      char status[11];  
};
```

ID	int id;
grad de prioritzare	char priority[7];
descriere	char *description;
status	char status[11];

Observație

Pentru a verifica validitatea sistemului, s-a raportat un incident fictiv care a fost salvat în memoria sistemului. Acesta are următoarele date:

- ID - 0
- grad de prioritzare - "low"
- descriere - "test incident"
- status - "solved"

Important

Incidentele vor fi salvate ca o listă dublu înlănțuită circulară cu santinelă. Incidentul fictiv de test poate juca rol de santinelă.

O intervenție reprezintă o acțiune de remediere a unui incident. În această temă, o intervenție se va defini printr-o **structură** ce conține: incidentul ce trebuie remediat (pointer la structură) și echipajul de intervenție responsabil de remediere (pointer la structură).

```
struct intervention {  
    detalii incident struct incident *incident;  
    detalii echipaj de intervenție struct unit *unit;  
};
```

detalii incident	struct incident *incident;
detalii echipaj de intervenție	struct unit *unit;

Important

Intervențiile vor fi salvate ca o listă dublu înlănțuită circulară cu santinelă.

E recomandată, NU necesară, definirea sistemului de gestionare a apelurilor de urgență 112 ca o structură de forma:

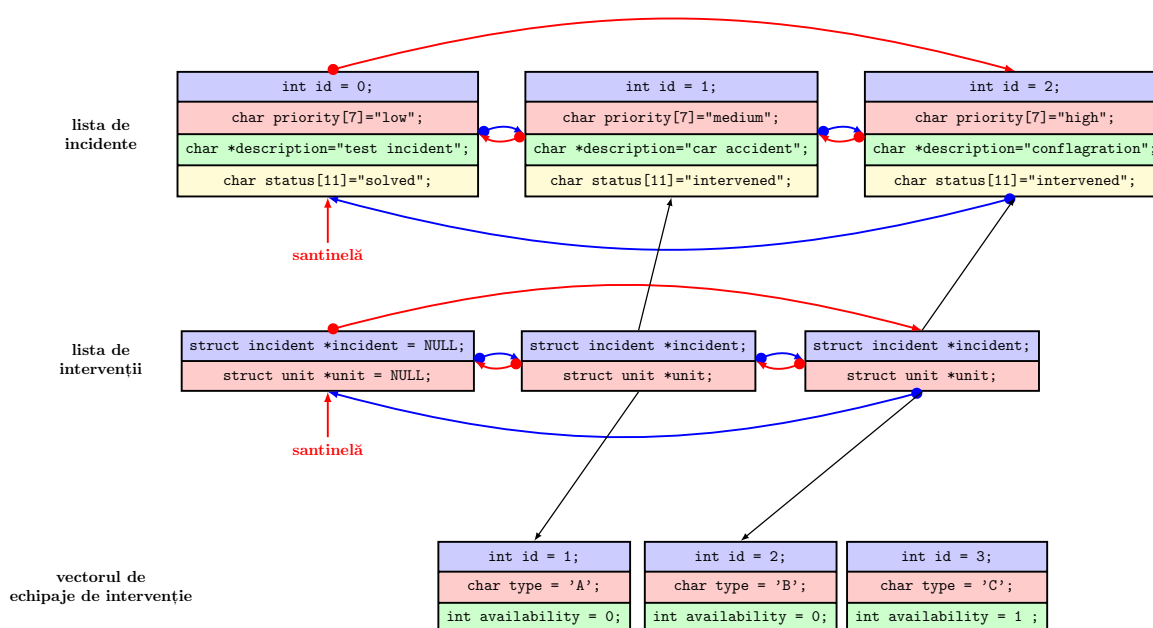
```

struct system {
    vectorul de echipaje de interventie
        lista incidente
        lista interventii
};

```

struct unit* units;
struct incident* incidents;
struct intervention* interventions;

O reprezentare completă a unui sistem este:



În ilustrația de mai sus este prezentat un sistem care urmărește remedierea a 2 incidente:

- incidentul cu ID = 1, care este remediat cu ajutorul echipajului de intervenție cu ID = 1
- incidentul cu ID = 2, care este remediat cu ajutorul echipajului de intervenție cu ID = 2

Observație

Inițial, în sistem se vor memora:

- vectorul de echipaje de intervenție citit din fișierul de intrare
- lista de incidente conținând santinela (incidentul de test)
- lista de intervenții conținând santinela

2 Prezentare operații posibile

Este necesară implementarea mai multor funcționalități ale sistemului.

2.1 ADD_INCIDENT <ID_incident> <priority> <description>

- când este raportat un incident nou, acesta va fi adăugat la **finalul** listei de incidente și va primi status-ul inițial "queued"
- în funcție de severitate (gradul lui de prioritizare), un incident nou raportat va trebui introdus într-o coadă de așteptare specifică pentru a putea fi ulterior remediat
- sistemul gestionează 3 cozi de așteptare: `queue_high`, `queue_medium` și `queue_low`; maparea [grad de prioritizare - coadă de așteptare] specifică este:
["high" - `queue_high`], ["medium" - `queue_medium`], ["low" - `queue_low`]

Detalii de implementare a cozilor de așteptare

În cozile de așteptare `queue_high`, `queue_medium` și `queue_low` **nu** se vor copia structuri, ci se vor salva **pointeri** către celule ale listei de incidente.

2.2 CHECK_UNITS_AVAILABILITY

- se impune menținerea unei cozi de disponibilitate a echipajelor de intervenție existente, denumită `queue_available_units`
- inițial, toate echipajele de intervenție sunt detectate ca fiind disponibile, prin urmare acestea vor fi introduse în coadă exact în ordinea în care sunt citite din fișierul de intrare
- sistemul va determina și va afișa numărul de echipaje de intervenție detectate disponibile la momentul execuției operației (practic, lungimea cozii `queue_available_units`)
- formatul de afișare este `Number of available units: <num_available_units>`

Detalii de implementare a cozii de disponibilitate

În coada de disponibilitate `queue_available_units` **nu** se vor copia structuri de date, ci se vor stoca **pointeri** către celule ale vectorului de echipaje de intervenție.

2.3 DISPATCH

- se verifică dacă există incidente care trebuie remediate în cozile de așteptare și echipaje de intervenție disponibile în coada de disponibilitate;
dacă ambele condiții sunt îndeplinite, se demarează o intervenție
- **regulă prioritizare:** un incident care se află într-o coadă cu grad inferior de prioritizare poate fi remediat doar dacă toate cozile cu grad superior de prioritizare sunt goale;
ordinea de remediere a intervențiilor este: `queue_high`, `queue_medium`, `queue_low`
- demararea unei intervenții: se extrage primul incident din cea mai prioritară coadă nevidă și primul echipaj din coada de disponibilitate a echipajelor de intervenție; apoi e adăugată o celulă la **finalul** listei de intervenții (incidentul va trebui marcat ca fiind "intervened", iar echipajul de intervenție trebuie marcat ca fiind "unavailable")

Observație

Se dorește păstrarea unui istoric al intervențiilor care au fost demarate, în eventualitatea în care se constată că o intervenție a fost demarată în urma unei greșeli de dispecerizare. Se impune folosirea unei **stive** pentru a ține evidența intervențiilor care au fost demarate. La fiecare DISPATCH efectuat cu succes, se va salva în stivă un pointer către intervenție.

2.4 UNDO_LAST_DISPATCH

- se amână remedierea unui incident, sistându-se ultima intervenție aflată în desfășurare, care trebuie eliminată din lista de intervenții;
practic, se extrag intervenții din istoricul de intervenții demarate (stivă) până se găsește o intervenție care nu a fost încă finalizată, intervențiile finalizate fiind ignorate
- incidentul asociat intervenției sistate este marcat ca având din nou status-ul "queued" și este plasat înapoi la **începutul** cozii sale specifice de așteptare;
echipajul asociat intervenției sistate este marcat ca fiind din nou "available" și va trebui introdus la finalul cozii de disponibilitate

2.5 SOLVED_INCIDENT <ID_incident>

- intervenția demarată pentru remedierea incidentului <ID_incident> a fost finalizată (**totuși, intervenția va fi păstrată în lista de intervenții**)
- incidentul <ID_incident> este marcat ca fiind "solved";
echipajul de intervenție asociat acestui incident este marcat ca fiind din nou "available" și va trebui introdus la finalul cozii de disponibilitate

2.6 SHOW_UNIT <ID_unit>

- se afișează detaliile echipajului de intervenție <ID_unit>, respectând următorul format:
Unit <ID_unit> is type <type['A'/'B'/'C']> and is [available/unavailable]

2.7 SHOW_INCIDENT <ID_incident>

- se afișează detaliile incidentului <ID_incident>, respectând următorul format:
Incident <ID_incident> has <priority["low"/"medium"/"high"]> priority,
the following description: "<description>" and is [queued/intervened/solved]
(mesajul este scris pe o singură linie)

2.8 SHOW_INTERVENTIONS

- se afișează detalii despre toate intervențiile din sistem, în ordinea în care au fost memorate în lista de intervenții, respectând următorul format:
Incident <ID_incident> was assigned to unit <ID_unit>,
and has the following status: "<status_incident>"
(mesajul este scris pe o singură linie)
- în cazul în care nu s-a demarat încă nicio intervenție, se va afișa mesajul:
No intervention has been initiated

Important

Mesajul de eroare este: "INVALID OPERATION! ERROR 404"

Acest mesaj se va afișa:

- în cazul operației DISPATCH, dacă nu există niciun incident ce trebuie remediat sau dacă nu există niciun echipaj de intervenție disponibil
- în cazul operației UNDO_LAST_DISPATCH, dacă istoricul de intervenții (stiva) nu conține nicio intervenție aflată în desfășurare
- în cazul operației SOLVED_INCIDENT <ID_incident>, dacă ID-ul este invalid sau dacă incidentul nu are status-ul "intervened" (un ID este invalid dacă nu are asociat un incident)
- în cazul operației SHOW_UNIT <ID_unit>, dacă ID-ul este invalid (un ID este invalid dacă nu are asociat un echipaj de intervenție)
- în cazul operației SHOW_INCIDENT <ID_incident>, dacă ID-ul este invalid (un ID este invalid dacă nu are asociat un incident)

3 Bonus

Se va acorda un bonus pentru implementările care nu au deloc memory leaks, adică erori la rularea testelor cu Valgrind.

4 Fișiere de intrare-ieșire

Datele se citesc din fișierul de intrare **tema1.in**.

Fișierele de intrare vor conține, pe prima linie, nr. echipajelor de intervenție folosite în test. Urmează apoi câte 1 linie pentru fiecare echipaj pe care se găsește ID-ul și tipul echipajului. Pe următoarea linie se află N, numărul de operații care vor fi executate de către sistem în test, urmat de N linii ce conțin operațiile descrise anterior.

Rezultatele se scriu în fișierul **tema1.out**.

5 Testarea temei

Temele trebuie să fie încărcate atât pe **checker-ul automat** cât și pe platforma **Moodle**, la secțiunea corespunzătoare. **NU** se acceptă teme trimise altfel decât prin **Moodle**.

O rezolvare constă într-o arhivă **zip** ce conține toate fișierele sursă alături de un **Makefile** ce va fi folosit pentru compilare și un fișier **README** în care precizați detaliile implementării.

Makefile-ul trebuie să aibă în mod obligatoriu regulile pentru **build** și respectiv **clean**. Regula **build** trebuie să aibă ca efect compilarea surselor și crearea binarului **tema1**.

Va fi pus la dispoziție un checker prin care veți putea testa corectitudinea implementării. De asemenea, acest checker va verifica existența fișierului README, dar nu și conținutul, acesta urmând să fie verificat ulterior, manual.

6 Exemple

Exemplu ADD_INCIDENT și SHOW_INCIDENT

tema1.in

```
(1) 2
(2) 1 A
(3) 2 B
(4) 4
(5) ADD_INCIDENT 1 high "fire in building"
(6) ADD_INCIDENT 2 medium "a car accident"
(7) SHOW_INCIDENT 1
(8) SHOW_INCIDENT 2
```

tema1.out

```
(1) Incident 1 has high priority, the following description: "fire in building"
and is queued
(2) Incident 2 has medium priority, the following description: "a car accident"
and is queued
```

Explicație

- Linia (1) indică existența a 2 echipaje de intervenție
- Liniile (2-3) descriu echipajele: echipajul 1 de tip 'A' și echipajul 2 de tip 'B'
- echipajele de intervenție sunt disponibile și se află în coada `queue_available_units`
- Linia (4) indică faptul că se vor efectua 4 operații
- Linia (5): se adaugă incidentul cu ID 1, prioritate "high", descriere "fire in building" și status inițial "queued"; incidentul este introdus și în coada `queue_high`
- Linia (6): se adaugă incidentul cu ID 2, prioritate "medium", descriere "a car accident" și status inițial "queued"; incidentul este introdus și în coada `queue_medium`
- Liniile (7-8): se afișează detaliile incidentelor 1 și 2

Exemplu CHECK_UNITS_AVAILABILITY și SHOW_UNIT

tema1.in

```
(1) 3
(2) 1 A
(3) 2 B
(4) 3 C
(5) 4
(6) CHECK_UNITS_AVAILABILITY
(7) SHOW_UNIT 1
(8) SHOW_UNIT 3
(9) SHOW_UNIT 5
```

tema1.out

```
(1) Number of available units: 3
(2) Unit 1 is type A and is available
(3) Unit 3 is type C and is available
(4) INVALID OPERATION! ERROR 404
```

Explicație

- 3 echipaje de intervenție sunt disponibile și se află în coada `queue_available_units`
- Linia (6): se verifică disponibilitatea curentă a echipajelor de intervenție => 3 echipaje
- Liniile (7-8): se afișează detaliile echipajelor 1 și 3
- Linia (9): nu există un echipaj cu ID 5, deci se afișează mesajul de eroare

Exemplu DISPATCH

tema1.in

```
(1) 2
(2) 1 A
(3) 2 B
(4) 8
(5) ADD_INCIDENT 1 low "noise complaint"
(6) ADD_INCIDENT 2 high "fire in building"
(7) ADD_INCIDENT 3 medium "a car accident"
(8) DISPATCH
(9) DISPATCH
(10) SHOW_INCIDENT 2
(11) SHOW_INCIDENT 3
(12) DISPATCH
```

tema1.out

```
(1) Incident 2 has high priority, the following description: "fire in building"
and is intervened
(2) Incident 3 has medium priority, the following description: "a car accident"
and is intervened
(3) INVALID OPERATION! ERROR 404
```

Explicație

- 2 echipaje de intervenție sunt disponibile și se află în coada `queue_available_units`
- Liniile (5-7): se adaugă 3 incidente, care trebuie adăugate și în cozile corespunzătoare
- Linia (8): se execută prima operație de dispecerizare; conform cu regula de prioritizare, se extrage incidentul 2 din `queue_high` (cea mai prioritară coadă nevidă) și, respectiv, echipajul 1 din coada de disponibilitate `queue_available_units`; e demarată intervenția, incidentul 2 primind status `"intervened"`, iar echipajul 1 devenind `"unavailable"`
- Linia (9): se execută a doua operație de dispecerizare; coada `queue_high` este acum goală, deci se extrage incidentul 3 din coada `queue_medium`, iar echipajul 2 e primul disponibil; se demarează intervenția, incidentul 3 primind astfel status-ul de `"intervened"`, respectiv echipajul 2 devenind `"unavailable"`
- Liniile (10-11): se afișează detaliile incidentelor 2 și 3
- Linia (12): se execută a treia operație de dispecerizare, care nu se poate realiza cu succes - deși mai există incidentul 1 în coada `queue_low`, nu mai este niciun echipaj disponibil; se afișează mesajul de eroare

Exemplu UNDO_LAST_DISPATCH

tema1.in

```
(1) 2
(2) 1 A
(3) 2 B
(4) 8
(5) ADD_INCIDENT 1 high "fire in building"
(6) ADD_INCIDENT 2 medium "a car accident"
(7) DISPATCH
(8) DISPATCH
(9) UNDO_LAST_DISPATCH
(10) CHECK_UNITS_AVAILABILITY
(11) SHOW_INCIDENT 2
(12) UNDO_LAST_DISPATCH
```

tema1.out

```
(1) Number of available units: 1
(2) Incident 2 has medium priority, the following description: "a car accident"
and is queued
```

Explicație

- 2 echipaje de intervenție sunt disponibile și se află în coada `queue_available_units`
- Liniile (5-6): se adaugă 2 incidente, care trebuie adăugate și în cozile corespunzătoare
- Linia (7): primul DISPATCH: incidentul 1 (coada `queue_high`) e asignat echipajului 1; se salvează intervenția în stivă
- Linia (8): al doilea DISPATCH: incident 2 (coada `queue_medium`) asignat echipajului 2; se salvează intervenția în stivă
- Linia (9): UNDO_LAST_DISPATCH: se anulează ultima intervenție care e nefinalizată (în acest caz, cea aferentă incidentului 2); incidentul 2 revine la status-ul de "queued" și este plasat la **începutul** cozii `queue_medium`, iar echipajul 2 devine din nou "available" și este introdus la finalul cozii de disponibilitate
- Linia (10): se verifică disponibilitatea curentă a echipajelor de intervenție => 1 echipaj
- Linia (11): se afișează detaliile incidentului 2 (care are acum status-ul "queued")
- Linia (12): UNDO_LAST_DISPATCH: se anulează ultima intervenție care e nefinalizată (în acest caz, cea aferentă incidentului 1); incidentul 1 revine la status-ul de "queued" și este plasat la **începutul** cozii `queue_high`, iar echipajul 1 devine din nou "available" și este introdus la finalul cozii de disponibilitate

Exemplu SOLVED_INCIDENT

tema1.in

```
(1) 2
(2) 1 A
(3) 2 B
(4) 8
(5) ADD_INCIDENT 1 high "fire in building"
(6) ADD_INCIDENT 2 medium "a car accident"
(7) DISPATCH
(8) SOLVED_INCIDENT 1
(9) UNDO_LAST_DISPATCH
(10) CHECK_UNITS_AVAILABILITY
(11) SHOW_INCIDENT 1
(12) SHOW_UNIT 1
```

tema1.out

```
(1) INVALID OPERATION! ERROR 404
(2) Number of available units: 2
(3) Incident 1 has high priority, the following description: "fire in building"
and is solved
(4) Unit 1 is type A and is available
```

Explicație

- 2 echipaje de intervenție sunt disponibile și se află în coada `queue_available_units`
- Liniile (5-6): se adaugă 2 incidente, care trebuie adăugate și în cozile corespunzătoare
- Linia (7): DISPATCH: incidentul 1 (din coada `queue_high`) este asignat echipajului 1; se salvează intervenția în stivă
- Linia (8): SOLVED_INCIDENT 1: intervenția aferentă incidentului 1 este finalizată; incidentul 1 primește status-ul de "solved", iar echipajul 1 devine din nou "available" și este introdus la **finalul** cozii de disponibilitate
- Linia (9): UNDO_LAST_DISPATCH: în stivă nu este nicio intervenție nefinalizată, așadar se va afișa mesajul de eroare (atenție: după operație, stiva ajunge să fie goală)
- Linia (10): se verifică disponibilitatea curentă a echipajelor de intervenție => 2 echipaje
- Liniile (11-12): se afișează detaliile despre incidentul 1 și despre echipajul 1

Exemplu SHOW_INTERVENTIONS

temal.in

```
(1) 3
(2) 1 A
(3) 2 B
(4) 3 C
(5) 9
(6) ADD_INCIDENT 1 high "fire in building"
(7) ADD_INCIDENT 2 medium "a car accident"
(8) ADD_INCIDENT 3 low "noise complaint"
(9) SHOW_INTERVENTIONS
(10) DISPATCH
(11) DISPATCH
(12) SOLVED_INCIDENT 1
(13) DISPATCH
(14) SHOW_INTERVENTIONS
```

temal.out

```
(1) No intervention has been initiated
(2) Incident 1 was assigned to unit 1, and has the following status: "solved"
(3) Incident 2 was assigned to unit 2, and has the following status: "intervened"
(4) Incident 3 was assigned to unit 3, and has the following status: "intervened"
```

Explicație

- 3 echipaje de intervenție sunt disponibile și se află în coada `queue_available_units`
- Liniile (6-8): se adaugă 3 incidente, care trebuie adăugate și în cozile corespunzătoare
- Linia (9): `SHOW_INTERVENTIONS`: nicio intervenție nu a fost încă demarată, așadar se va afișa mesajul corespunzător
- Linia (10): `DISPATCH`: incidentul 1 (din coada `queue_high`) este asignat echipajului 1; se salvează intervenția în stivă
- Linia (11): `DISPATCH`: incidentul 2 (din coada `queue_medium`) e asignat echipajului 2; se salvează intervenția în stivă
- Linia (12): `SOLVED_INCIDENT 1`: intervenția aferentă incidentului 1 este finalizată; incidentul 1 primește status-ul de "solved", iar echipajul 1 devine din nou "available" și este introdus la **finalul** cozii de disponibilitate
- Linia (13): `DISPATCH`: incidentul 3 (din coada `queue_low`) este asignat echipajului 3; se salvează intervenția în stivă
- Linia (14): `SHOW_INTERVENTIONS`: se afișează toate intervențiile din lista de intervenții, cu status-ul curent al fiecărui incident

7 Punctaj

O temă perfectă valorează 100 de puncte. 95 de puncte se vor acorda exclusiv pentru teste și 5 puncte pentru README. Vor exista atât teste simple care verifică o operație specifică, cât și teste complexe în care există majoritatea operațiilor.

În urma corectării manuale, punctajul acordat de către checker-ul automat poate fi scăzut cu până la maximum 15 puncte pentru coding style. De asemenea, conținutul fișierului README va fi verificat manual, iar punctajul obținut pentru README poate fi diminuat.

Punctajul pe teste este următorul:

Cerința	Punctaj
Operații ADD_INCIDENT	15 puncte
Operații CHECK_UNITS_AVAILABILITY	10 puncte
Operații DISPATCH	15 puncte
Operații UNDO_LAST_DISPATCH	15 puncte
Operații SOLVED_INCIDENT	10 puncte
Operații SHOW_UNIT / SHOW_INCIDENT	10 puncte / 10 puncte
Operații SHOW_INTERVENTIONS	10 puncte
README	5 puncte
BONUS (testat cu valgrind)	20 puncte
TOTAL	120 puncte

Atenție!

- Orice rezolvare care nu conține structurile de date specificate **NU** este punctată.
 - **echipajul de intervenție** va fi implementat ca o structură ce conține: un nr. întreg, un caracter și încă un nr. întreg cu rol de flag
 - **incidentele** vor fi salvate într-o listă dublu înlănțuită circulară cu santinelă care conține: un nr. întreg, un șir de 7 caractere, unul de dimensiune variabilă și altul de 11 caractere;
 - **intervențiile** vor fi salvate într-o listă dublu înlănțuită circulară cu santinelă care conține pointeri către echipajul de intervenție și către incidentul remediat;
 - **Pentru orice altă soluție se anulează punctajele checker-ului automat și nu se vor primi puncte pentru README sau coding style.**
- Toate echipajele de intervenție folosite în teste vor fi citite din fișierul de intrare. Vor exista maxim 50 de echipaje de intervenție și le puteți salva în orice mod doriți (vector de structuri, listă, etc.).
- Descrierea unui incident va fi o propoziție, furnizată între ghilimele.
- **Pentru descrierea unui incident trebuie să alocați exact atâta memorie cât este necesară ($\text{strlen}(\text{string-ul dintre ghilimele}) + 1$).**
- **Folosirea variabilelor statice și/sau globale este interzisă!**
- Temele vor fi punctate doar pentru testele care sunt trecute pe checker-ul automat.
- Nu lăsați warning-urile nerezolvate, deoarece veți fi depunctați.
- **Tema este muncă individuală! Toate soluțiile trimise vor fi verificate folosind o unealtă pentru detectarea plagiatului.**